Answer Sheet

Write your answers on this sheet. Exam questions & instructions can be found on subsequent sheets.

Name:

Student ID number:

Answers to knowledge questions:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A																				
B																				
C																				
D																				

Answers to hands-on questions:

	21	22	23	24	25	26	27	28	29	30
A										
B										
C										
D										

31

valM	
R[rsp]	
R[rA]	
PC	

32

VPN	
TLB index	
TLB tag	
TLB hit	
Page fault	
PPN	

33

A	
B	

34

	SU	ST	PU	PT
SU				
ST				

Answers to analytical questions:

35

36

End of answer sheet.

This page was intentionally left blank.

Exam

Willard Rafnsson
IT University of Copenhagen

January 10, 2025

Course name: Operating Systems and C

Course codes: BSOPSYC1KU and KSOPSYC1KU

Aids allowed: Written and printed books and notes. Pen.

Duration: 5 hours.

Write all of your answers on the provided answer sheet.

The grade is determined based on an overall assessment of all answers to the exam questions.

A blank answer yields 0 points. A correct answer yields a fraction of the available points in the question. An incorrect answer yields negative points, smaller the more available answer options there are.

For each option in multiple-choice questions, write **T** or **F** (true or false) in the appropriate field on the answer sheet. A multiple-choice question can have 1 to 4 correct options.

Do not hand in pages 1 to 12; we will not read them.

Good luck.

Knowledge Questions

Question 1

Where does the first machine code instruction that your computer executes, upon being powered on, reside?

A. RAM

B. ROM on the motherboard

C. CPU

D. boot loader on disk

STEPS:

1. *Zapni PC*
2. *CPU zacne vykonavat kod z ROM (flash pamat na motherboarde): old BIOS, now UEFI – CPU nema na vyber, musi to spravit*
3. *Firmware spravi POST (PowerOnSelfTest) – test:cpu,ram,gpu, periferie. – problem=beep a zistit z ktoreho zariadenia sa bude bootovat (najde bootloader)*
4. *Spusti BOOTLOADER z disku: skopiruje kernel do RAM. Skoci na entry point kernelu. Pripravy parametre jadra, pamat mapu, ODOVZDA KONTROLU CPU*
5. *Kernel RUN from RAM – OS a kernel ma kontrolu nad CPU. Kernel: Zapne paging, virt. Memory, drivers, init schedulerel, mountne root filesystem. Spusti sa 1. Process PID=1 – system*

Question 2

What is the job of a boot loader?

A. power-on self-test. >firmware does this

B. load the OS kernel image into RAM.

C. pass control to the OS initialization code.

D. start the first process. >*Kernel does this*

Question 3

Which of the following does the CPU use to determine what code to execute, upon receiving an interrupt?

A. PC >*program counter urcuje aktualnu instrukciu*

B. IRQ>to je len signal/identifikator interruptu. Nerozhoduje aky kod sa vykona

C. IDTR>*register ktory ma pointer na IDT (table kde je adresa handlera)*

D. interrupt number> PRESNE TOTO POVIE KTORY KOD SA VYKONA

Interupt=signal na stop aktualneho threadu. Moze byt timer (napr each 20ms) alebo mys, klavesnica…

1. *Niekto vyvola interrupt (moze byt timer, ale teraz napr. Klavesnica) posle PIC-Programmable interrupt controller signal ze chce pozornost.*
2. *PIC rozhodne prioritu a masku a posle INTERRUPT REQUEST do CPU*
3. *CPU posle naspat INTA (acknowledge) pomocou DATA BUS asi*
4. *PIC vytvori INTERRUPT VECTOR (napr. 33(0x21). Tento vector je kod ktory CPU nevie. Musi si ho najst v RAM.*
5. *CPU ma svoj register IDTR ktory ma pointer na table v RAM kde su ulozene instrukcie pre kazdy takyto INTERRUPT VECTOR. Najde prisluchajuci kod a spusti ho. TO SA VOLA HANDLER pre*

tento konretny vector.

6. *Tento HANDLER precita udaje ulozene z HW registry a ulozi do OS buffer a posle EOI (end of interrupt) a PIC vtedy oznaci tento konretny IRQ ako ukonceny.*

Question 4

What causes a running process to eventually yield control, such that another process can run in its place?

A. the running process itself. *>Process sa yieldne sam len ak musi na nieco cakat. Teda ak je blocked*

B. the process that will run in its place. *> procesy sa nevedia vypnut navzajom*

C. scheduler. *>Scheduler len povie ktory ma momentalne bezat na zaklade priotit*

D. timer interrupt. *>Time interrupt kazdych X ms vypne running thread, aby mohol bezat iny.*

Question 5

What must take place during a context switch?

A. an interrupt handler is executed. *>Ano, lebo sa posle interrupt. Bez toho neprestane thread bezat*

B. process-specific register values are saved to disk. *>Not to disk, but to stack*

C. scheduler chooses which process executes next. *>Yes*

D. process is put into the "blocked" state. *>Nemusi ist, pokial nastal context switch kvoli time interrupt, tak proste ide do waiting listu.*

Contex switch: prepnut CPU z jedneho threadu na iny.
Nastane ak:

1. *Timer interrupt: OS chce spustit iny process*
2. *Process ide do blocked status alebo sa ukonci (lebo skoncil alebo boll killnuty -vymazany)*
3. *Vyssia priorita: do waiting listu pride thread s vyssiou prioritou: Os stopne running thread.*

Vtedy sa ulozi stav stareho procesu: Registry, PC a stack
A nacita sa stav noveho.

Question 6

Rule 3 of Multi-Level Feedback Queues states that when a job enters the system, it gets top priority. Now consider the following edge case: P is a program that creates a new process that also executes P, then orphans the new process, and then terminates. Suppose P is executed, and that the resulting process is the sole process with topmost priority on the system. Which of the following rules prevent starvation of lower-priority processes?

A. rule 1 (if priority(A) > priority(B), then A is scheduled). *> toto neriesi starvation*

B. rule 2 (if priority(A) = priority(B), A and B are scheduled in round-robin fashion).*>riesi!*

C. rule 4 (when job uses its CPU time allotment, move job to lower queue).*>neriesi starvation*

D. rule 5 (after time S, move all jobs to topmost queue). >riesi!

Multi-Level Feedback queues: Thready vo waiting liste maju priority a su ulozene do jeden z queue podla priority. Napr. Q0 (top priority), Q1 middle priorirty a Q2 low priority.

Thread pri vstupe ide do top priority. Potom sa mu meni priorita a tym padom moze ist do inych queue. Rychly process je v top queue, ak je dlhy tak ide do nizsich – ak vycerpa casovy kvantum (kolko casu ma vyhradene na run). PO URCITOM CASE IDU VSETKY NASPAT DO TOP queue, aby sa zasa sa riesilo starvation. V kazdej queue potom funguje ROBIN (teda napr bezi process1, potom 2, potom 3 hoci maju rovnake priority, idu postupne…)

Question 7

What prevents a process running in user-mode from **writing** to kernel-space memory?

A. monitor.*>Monitor nepreventuje. On len nastavuje/kontrolje. Proste toto sa nikdy nestane*

B. supervisor bit (in page table entry).*>Ano, to nam povie ci stranke je pristupna aj z USER modu.*

C. current privilege level (in the CPU).*>Ano, to nam povie ci sme USER alebo KERNEL mod.*

D. MMU.*>toto len preklada virt. Na fyzicku adresu, nepreventuje to proti tomu. To sa deje az potom*

MemoryWrite question: Tu to riesi supervisor bit – tento bit len povie ci je tato adresa pristupna aj z user modu.

Current CPU privilig – on hovori ci sme USER alebo KERNEL mod. Potom sa porovnava cpu privlige lvl a supervisor bit ci mame pristup.

Question 8

What prevents a process running in user-mode from entering kernel-mode without passing control to kernel-space code?

A. instruction set. *>ANO, instrukcie su rozne. Niktore iba z user modu, niektore len z kernel modu. Z user modu nemoze thread vykonat zmenenie MODU.*

B. monitor.*>cez neho musi ist kazdy PRISTUP. On neriesi thready atd.. len pristupy*

C. namespaces.*>Namespaces izoluju zdroje a nie zmeny stavov*

D. virtual memory.*>Izoluju memory a nie zmeny stavov*

Process changing MODE alone without OS problem.

Question 9

Which of the following statements about file systems are true?

A. a file can have zero links to it.*>ak 0 lins, tak sa vymaze*

B. a file can have more than one link to it.*>ano, superlinks*

C. the file system keeps track, for each process, which files it is using.*>keepuje ktore files su otvorene, ale nie per process ale celkovo – jedna velka tabulka.+curzors*

D. the process keeps track of its cursors in each file it is using.*>cursory su len vo velkej table, nie per process.*

Question 10

How does a multicore CPU execute a parallel program?

A. instruction set lets the programmer specify on which core an instruction should run.*>vobec*

B. one core directly signals another core to start running a task.*>core nic nehovori ziadnemu*

C. a task is created by means of a system call.*>Ano, scheduler atd…*

D. cores pick tasks from a shared pool of tasks in memory.*>Well, toto si myslim ze je picovina. Pretoze scheduler vyberie kto ma bezat a nie core, ale ok no.*

Question 11

Each thread has its own...

A. stack

B. heap>*zdiela s processom*

C. global values>*global zdiela, local nezdiela*

D. text data>*text data je code processu. Ten zdiela.*

Question 12

Which one of the following addresses is 8-byte aligned?

Hexa ma 0-15 values. Teda 0-F. 8byte musi byt nasobok 8. cize 8,16(ale 16 je zas 0.) takze bud 8 alebo 0 na konci.

Alligned znamena z eje nasobok 8.

A. `0xEDE7`

B. `0xEDE0 >0 takze nasobok.`

C. `0xEDE4`

D. `0xEDE6`

Question 13

Consider a 4-way set associative cache (E = 4). Which one of the following statements it true?

4way == E=4 znamena ze ma 4 ways. (NIE SETS). Cize tu vieme len to ze su 4.

A. The cache has 4 blocks per line. >*per chache line nemame bloky ale page.*

B. The cache has 4 sets per line. >*nevieme info o setoch*

C. The cache has 4 lines per set.>*na jeden set su 4 cache-lines. Sedi!*

D. The cache has 4 sets per block.>*nemame info o setoch*

Question 14

Which of the following can be utilized to eliminate race conditions?

A. lock instruction prefix. >*Lock instruction spravy viacej instrukcii atomickych. Takze ano*

B. cache coherence protocol. >*Toto len povie inemu procesu ze memory ktoru ma v chache sa prepisala*

C. bus arbiter.>*ani neviem co toto je*

D. compare and swap.>*Atomicka instrukcia, ANO*

Question 15

Which of the following are valid uses of memory barriers?

Memory barrier je mechanizmus, kt. Zabrani CPU a kompilatoru aby preusporiadali operacie.

A. protect process memory from being accessed by other processes.>*Nema to nic s pamattou*

B. prevent instruction reordering.>*Ano presne toto!*

C. lock-free programming. >*lock free znamena ze nebudeme mat ziadne locky v kode, takze musime*

pouzit meory bariesr

D. containers.

Question 16

Which of the following are benefits of register renaming?

A. eliminates false data dependencies.*>Presne toto, pretoze false data dep. Vznikne ak thready mozu bezat bez cakania, ale nemozu lebo chcu pouzit rovnaky register name. KOKOTINKA*

B. reveals more instruction-level parallelism.*>ano, viac instrukcii sa moze vykonat, len v inych registroch*

C. facilitates out-of-order execution *>ano, musi to splnat toto*

D. helps better utilize the resources of a superscalar processor.*>Tiez ano, lebo viac instrukcii sa moze diav vo viacerych vypocetnych units.*

Question 17

Which of the following statements regarding memory aliasing are true?

Memory aliasing: ak viac pointerov ukazuje na rovnake miesto v pamati

A. memory aliasing can prevent the compiler from optimizing away some memory I/O.*>Ano, lebo compiler nevie ci sa ta pamat zmenila alebo nie atd…*

B. you must under no circumstance use two different pointers that can specify the same location.*>kokotina, mozes*

C. introduce local variables to eliminate the possibility of aliasing.*>Ano lebo sa nebude priamo riesit pamat, ale len variable*

D. keep pointers in separate scopes to prevent the possibility of aliasing.*>No toto tiez nepomoze, ak mam viac tych pointov v rovnakom scope.*

Question 18

Why is software prefetching generally discouraged today?

A. it costs an instruction.*>Ano, ale toto nieje hlavny dovod*

B. you might negatively impact performance.*>Ano, mozu sa vyhodit dolezite udaje z cache, a aj tak sa nemusi pouzit ten prefetch*

C. your code performance will be less portable.*>Intel, amd maju ine velkosti cache, latencia architekturu takze pre kazdy processor/brand treba iny kod.*

D. software prefetching is error-prone.*>Nie, on nikdy nesposoby logicke chyby alebo nieco, crash.*

Question 19

How are Linux Security Modules implemented in Linux?

A. by means of access control lists (ACLs).*>Toto je vo file-system – riesi pristup k suborom.*

B. a security check is made before any security-sensitive operation. *> ANO presne*

C. with namespaces and cgroups. >tto izoluje a nie security

D. hooks invoke the LSM kernel module to resolve security check.*>Ano*

Question 20

When creating a container, what are namespaces used for?

A. Isolation *>Ano*

B. Provisioning > nie, vytvorit filesystem, config, siet users.. proste nachystat prostrenie pre app

C. Virtualization > *nie, emulacia / abstrahovanie celeho OS napr. VM*

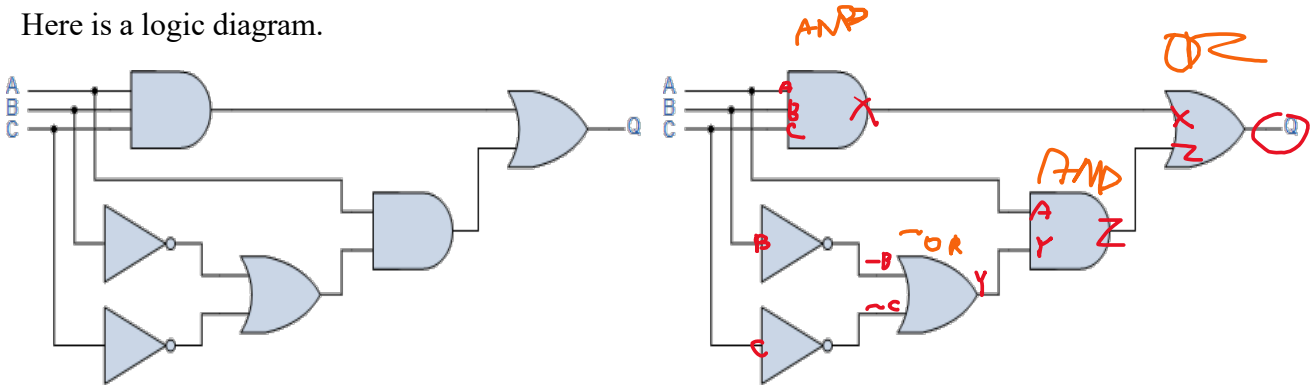
D. Hardening > *zvysovanie bezpecnosti, zmensovanim attack surface – napr. Nemaspaces, readonly filesystem…*

End of knowledge questions. Exam continues on the next page.

Hands-on Questions

Question 21

Here is a logic diagram.



(The inputs of the circuit, top-to-bottom, are A, B and C. The output is Q.)

X.= A&B&C

Y = (~B | ~C)

Z = Y & A
Z = (~B | ~C) & A

Q = Z | X
Q = ((~B | ~C) & A) | A&B&C)
Q = A& ((~B | ~C) | (B&C))

(~B | ~C) <==> ~(B & C)
Q = A& (~(B & C) | (B&C))

(~(B & C) | (B&C)) → *~A | A = always 1. 0 | 1 is always 1.*
Q = A & 1
Q = A

Which of the following Boolean expression are equivalent with Q (the output of the circuit)?

A. `A&B|~C`

B. `A&(~C|~B) | C&B&A`

C. `A&( B&C | ~(B | C) ) > yes boss`

D. `A > Yes boss`

Question 22

Consider the following code.

```
int x[2][128];
int i;
int sum = 0;
for ( i = 0 ; i < 128 ; i++ ){
  sum += x[0][i] * x[1][i];
}
```

X je array ktore ma 2 prvky, a v kazdom prvku je 128 intigerov.

X[0] = int1,int2,int3...int128 a zaroven X[1] = int1,int2.

Takze kazdy tento array ma 128(pocet bajtov pre prvok int=4) = 512B per ARRAY.*

Assume we execute this under the following conditions.

- `sizeof(int)` = 4.
- array `x` begins at memory address `0x0` and is stored in row-major order.
- the cache is initially empty.
- the only memory accesses are to the entries of array `x`; all other variables are stored in registers.

Assume that the cache is 512 bytes, direct-mapped, with 16-byte cache blocks. What is the miss rate?

Kedze cache ma 512B tak v podstate moze byt nacitany len X[0] alebo X[1] naraz.

V programe spocitavame prvky z oboch naraz, takze kazdy step sa musi nacitat ARRAY[0], precitat prvok, a vyhodit cache, nacitat array [1] atd.. TAKZE kazdy iteration sa robi miss.

A. 0%

B. 10%

C. 50% alebo D. 100. JE TO STO

Postup na cache miss-rate úlohy

1) Urob si parametre cache

- **Cache size** = 512 B
- **Block size** = 16 B
- **#lines** = 512 / 16 = **32 liniek**
- **Ints per block** = 16 / 4 = **4 inty**

Direct-mapped znamená:

- každý blok z RAM ide presne do **jednej** cache linky podľa indexu.

2) Napíš si adresu každého prístupu

Pole `x[2][128]`, row-major, `sizeof(int)`=4, začína na adrese 0.

- `x[0][i]` má adresu:
$A0(i) = 4i$

- `x[1][i]` je o celý prvý riadok ďalej. Prvý riadok má 128 * 4 = **512 B**:
$A1(i) = 512 + 4i$

3) Prelož adresu na "block number" a "cache index"

- **block number** = `address / 16`
- **index** = `block_number mod 32` (lebo 32 liniek)

Pozri konflikt:

- pre `x[0][i]`: block = ⌊(4i)/16⌋ = ⌊i/4⌋
- pre `x[1][i]`: block = ⌊(512+4i)/16⌋ = 512/16 + ⌊i/4⌋ = 32 + ⌊i/4⌋

Indexy:

- index0 = (⌊i/4⌋) mod 32
- index1 = (32 + ⌊i/4⌋) mod 32 = (⌊i/4⌋) mod 32

Indexy:

- index0 = (⌊i/4⌋) mod 32
- index1 = (32 + ⌊i/4⌋) mod 32 = (⌊i/4⌋) mod 32

👉 **Sú rovnaké.**
Takže `x[0][i]` a `x[1][i]` sa mapujú do **tej istej cache linky** → budú sa vyhadzovať.

4) Simuluj správanie (stačí pár krokov, potom uvidíš vzor)

Iterácia `i=0`:

- access `x[0][0]` → cache empty → **miss**, natiahne blok `x[0][0..3]`
- access `x[1][0]` → mapuje sa do tej istej linky → **miss**, vyhodí blok `x[0][0..3]`, natiahne `x[1][0..3]`

Iterácia `i=1`:

- `x[0][1]` by teoreticky mohol byť hit (je v tom istom bloku ako `x[0][0]`), **ALE** ten blok bol vyhodený prístupom na `x[1][0]` → **miss**
- `x[1][1]` zas vyhodí `x[0]` blok → **miss**

👉 Takto to ping-ponguje furt.

Výsledok pre tento konkrétny príklad

- Počet prístupov do pamäte: 128 iterácií × 2 = **256**
- Každý prístup je miss (konflikt v direct-mapped cache) → **256 miss**
- **Miss rate = 256/256 = 1 = 100%**

Question 23

Consider an image processing scenario, where each image is of size `h * w * c`, where `h` is the image height, `w` its width, and `c` the number of channels (i.e. spectral bands) in the image.

(The image could have the typical three spectral bands (red, green and blue), or it could be a multi-spectral image with tens, even hundreds of spectral bands, as is the case in e.g. deep-space imaging).

Suppose you wish to scale each channel `k` by a constant `sca[k]`, and add the resulting channels together, producing the single-channel image `out`. Here is a code sketch that achieves this.

```
for ( int i = 0; i < h; i++ ) {
  for ( int j = 0; j < w; j++ ) {
    double o = 0.0;
    for ( int k = 0; k < c; k++ ) {
      o += img[ ??? ] * sca[ k ];
    }
    out [ i, j ] = o;
  }
}
```

We have left unspecified how `img` is indexed (indicated by the `???`).

How should `img` be stored to maximize cache efficiency?

Well since first loop is i==H, then j==W, and then k==C, then

[h][w][c] is correct

A. `h*w*c`

B. `w*h*c`

C. `c*h*w`

D. `c*w*h`

Question 24

Now suppose you use GPU processing with CUDA to improve the performance of the image scaler from before. After computing `i` and `j` (using `threadIdx`, `blockIdx`, and `blockDim`), the remainder of the body of the CUDA kernel is as follows.

```
double o = 0.0;
for ( int k = 0; k < c; k++ ) {
  o += img[ ??? ] * sca[ k ];
}
out [ i, j ] = o;
```

Again we have left unspecified how `img` is indexed (indicated by the `???`).

How should `img` now be stored to maximize cache efficiency?

Well cuda can do the same opration on all pixels in the same time

(because it will load it and do it once), TAKZE lepsie ak vypocitame K pre

*kazdy pixel h*w naraz, takze c moze ist na zaciatok, lebo nemusime ho*

citat za sebou v loope, ale to spravy coda naraz.

A. `h*w*c`

B. `w*h*c`

C. `c*h*w >this`

D. `c*w*h >this`

Question 25

What kind of locality does the code snippet in **Question 23** exhibit?

A. spatial locality of data with respect to elements of `sca`.

B. temporal locality of data with respect to elements of `sca`.

C. spatial locality of instructions with respect to the innermost loop body.

D. temporal locality of instructions with respect to the innermost loop body.

*For k, k<c,.. o+= img sca*k. tato slucka sa opakuje pre kazdy i,j pixel.*

a. *Spatial loc (pouzivanie znovu) element of sca. Ano, pouzivam I,j*

b. *Temporal (pouzivame nasledujuce) tie iste v opakovanom case za sebou*

c. *Spatial to innermost: ano, par instrukcii ale sekvencne v pamati – nacitane v 1 bloku*

d. *Sptil sa vykonaju zas a zvnova*

LOCALITY sa tyka aj instrukcii!!! Aj DAT!!!

SCA – DATA

InneR LOOP – instrukcie su blizko seba/za sebou casto

Question 26

Consider the following C source file `t.c`.

```
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
int s = 0;
void *add(void* arg) {
  int a = s + (int)arg;
  usleep( ( (float)rand() / RAND_MAX ) * 10 );
  s = a;
}
int main() {
  pthread_t tid2;
  pthread_t tid3;
  srand(time(NULL));
  pthread_create( &tid2, NULL, add, (void*)2 );
  pthread_create( &tid3, NULL, add, (void*)3 );
  pthread_join(tid2, NULL); // thread 2 join
  pthread_join(tid3, NULL);
  printf("%d\n", s);
}
```

It compiles (with warnings) with the following command

```
gcc -pthread -o t t.c
```

Upon executing this program (`./t`), which of the following are possible outputs of the program?

Vzdy pozeraj na moznosti.

A. 0 > *obe thready musia urobit add funkciu, takze nestane ze bude 0*

B. 2 >*stane sa ak runne create_tid2 a potom print*

C. 5 >*stane sa ak sa runne tid2 ahned tid3, tym padom a=2+3 a s=5. a potom print*

D. 10 >*Nema sa ako stat kedze add sa vola len 2x*

Question 27

Which of the following statements about the code snippet from **Question 26** are true?

A. the program can enter a livelock.*>No, musi tam byt nejake if(trylockA) if(trylockB) ...*

Livelock: ak thready pustaju stale iny thread, takze sa nevykona ziadna robota

B. protecting access to `s` by a mutex will prevent race conditions.: only 1 thread can access to S in the time, AK pridame napr. Ptherad_mutex_lock pred ty mako robime a = s+arg, …s=a a potom thread_unlock, tak bude to ok.!

C. moving the line marked “thread 2 join” 1 line up will prevent race conditions.*>Ak create TID2 a hned za nim ho joinem tak thread2 musi skoncit kym sa vobec spravy thread3. Riesi to problem, ale hlupo*

D. the critical region consists of all instructions that can be encountered after thread creation*>kriticka sekcia je v add funkcii*

Question 28

We consider code that adds numbers in an array in a particular way. Assume the following declarations.

```
unsigned short n;
unsigned short num[n];
unsigned long sum;
```

The `num` array is filled with numbers (e.g. from standard input). Finally, the following "sum code" is run on the array.

```
// sum code
for (unsigned short i = 0; i < n; i++) {
  if (num[i] >= 256)
    sum += num[i];
  }
}
```

This sum code runs faster if `num` is sorted. Why is that?

(On a particular machine, for `n` = 32768, the sum code took 115 microseconds to run with `num` filled with random numbers, and 19.3 microseconds with `num` sorted).

A. branch prediction.>*ak je num zoradene, tak zo zaciatku mame stale misprediction, ale po case, ked bude>=256, tak uz vsetky predictions budu spravne az po koniec.*

B. caching.>*ideme seqvencne cez num, takze cache je stale rovnaka – nemeni poradie loadingu*

C. alignment.> *unsigned short n, short num n a long sum su v pamati stale rovnako po sebe, nemeni sa nic tym ze sortnes*

D. code motion.>*code motion je presunutie vypoctov mimo slucku aby rycheljsie sla slucka. Nic take sa pri sorting nekona*

Question 29

Here is a short program written in Y86-64.

```
0   irmovq $0xFF, %rax
1   xorq %rdi, %rax
2   je i
3   rrmovq %rdi, %rax
4   halt
5 i:irmovq $2, %rax
6   addq %rdi, %rax
7   halt
```

Assume `%rax` = `0xcafe`, and that the program is running on PIPE hardware (including forwarding- and pipeline-control-logic).

Hint: It may be helpful to simulate (on paper) this program running on the above-stated hardware, i.e. to fill out a figure along the following.

	pipeline				
Cycle	F	D	E	M	W
0	0				
1	1	0			
2	2	1	0		
3	. . .				

The program has one or more hazards in it. In which cycle does the pipeline encounter its first hazard?

A. cycle 3 >*nie lebo mame forwarding atd.*

B. cycle 4 > nie lebo…

C. cycle 5 > ANO

D. cycle 6

Cycle	F	D	E	M	W	Info
0.	0					
1.	1	0				
2.	2	1	0			0 je v E -> vyraba 0xFF. 1 je v D a ziti ze potrebuje RAX a RDI
3.	3	2?	1	0		1 je v E a potrebuje RAX. Kedze 0 uz bola minuly cycle v E, tak mozme forwardovat hodnotu (kedze to bola raw hodnota). Fetch 3: aj ked 2 este nebula v E faze – BRANCH PREDICION na NOT TAKEN
4.	4	3	2	1	0	2 sa vyhodnocuje – teda zistime ci guess bol good or bad.
5.						Ak zly prediction tak flush 4 a 3. Takze tu SA RIESI TEN HAZARD. Nemusi to byt problem ak sme tipli OK ale moze!
6.						

ReadAfterWrite hazard: a. pokial je hodnota raw$, tak musel byt ins. V E faze a teraz je v M.

b. ak je to ()nacitavanie napr. M[0x84] tak musel byt v M a teraz je v W. – LOAD USE

Control hazard: branch prediction: CMP a JE, pri fetchingu sa rozhodne NOT TAKEN, aby sa dalsie kolo mohla loadnuj dalsia tipnuta instrukcia. CMP sa vyhodnoti v E, a dalsi cyklus sa riesi PROBLEM.

Question 30

What kind of hazard did the program in **Question 29** encounter first?

A. load/use hazard

B. read-after-write

C. branch misprediction > Presne toto

D. ret

Question 31

Consider the following Y86-64 instruction.

```
POPQ %rdi
```

Show how this instruction executes, by filling in values in fields **valM**, **R[rsp]**, **R[rA]**, and **PC**.

Assume SEQ hardware. At the time this instruction is executed, `%rdi` has value `0x42`, `%rsp` has value `0x62fe`, `PC` has value `0x20241e`, and the words stored in memory near `%rsp` are as follows.

```
M8[ 0x62f0 ] = 0xabacadabacab0000
M8[ 0x62f8 ] = 0x0badf00dcafe0000
M8[ 0x6300 ] = 0x0000feedfacebeef
```

The values you fill into the fields should be in hexadecimal notation. Please leave out the leading `0x`.

Inštrukcia	Dĺžka
halt	1 B
nop	1 B
rrmovq	2 B
popq rA	2 B
pushq rA	2 B
irmovq V, rB	10 B
jXX Dest	9 B
call Dest	9 B
ret	1 B

Takze jednoducho. POPQ %RDI – popne zo stacku 8B na ktore ukazuje RSP.

V provom rade si urobim stack values. Teda rozpisem vsetky adresy a ich samostatne values. LITTLE INDIAN.

V podstate `M8[ 0x62f0 ] = 0xabacadabacab0000 ukazuje hodnty pre M8 adries, teda 16/8 = 2B pre kazdy. Takze pre 62f0,62f1,...,62f7.`

62f0 = `0xabacadabacab0000`. `Ale LITTLE INDIAN, takze zoadu 2 pre tuto adresu .(najnizsia adresa je vzdy vzadu.)`

62f1 = 00

62f2 = ab

62f3 = ac

62f4 = ab

62f5 = ad

62f6 = ac

62f7 = ab

62f8 = `0x0badf00dcafe0000`

62f9 = 00

62fa = fe

62fb = ca

62fc = 0d

62fd = f0

62fe = ad. *<<< RSP tu ukazuje. RSP ukazuje na posledny prvok (najnizsi adresovo) z 8.*

62ff = 0b

6300 = `0x0000feedfacebeef`

6301 = be

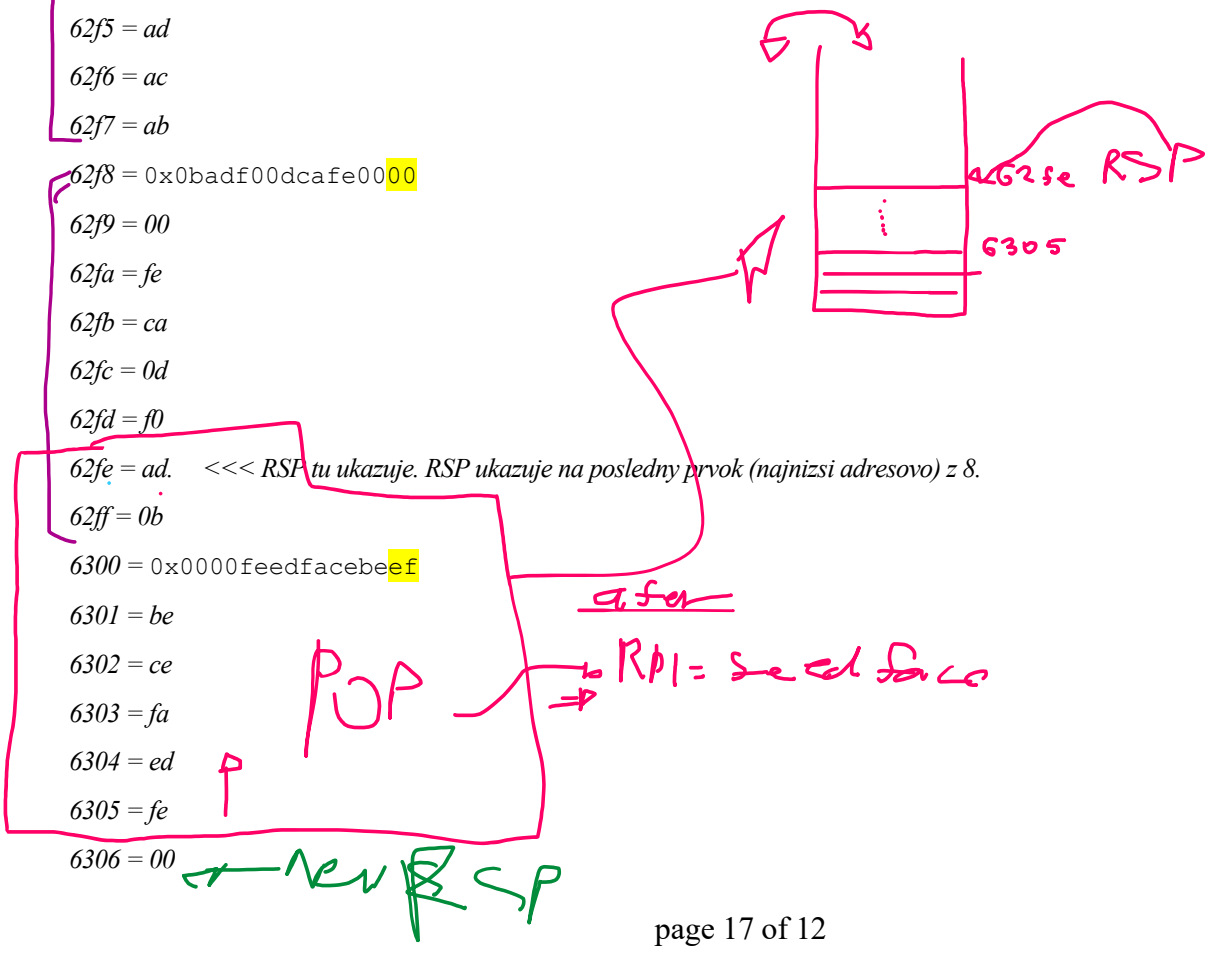
6302 = ce

6303 = fa

6304 = ed

6305 = fe

6306 = 00



6307 = 00'

Question 32

Consider the following memory system.

- page size is 256 bytes.
- virtual addresses are 14 bits long.
- physical addresses are 12 bits long.
- there is a 3-way TLB with 4 sets.
- memory is byte-addressable.

The memory system thus has the following system parameters: **VPO** = 8, **PPO** = 8, **VPN** = 6, **PPN** = 4.

Suppose the page table contains the following data.

VPN = 6 , VPO = 8 = 14bits as virtual
PPN = 4, PPO = 8 = 12bits as physical
3way TLB (translation l.buffer) cache to znamena ze ma 3 'bloky' a 4 sety.

VPN	PPN	Valid
10	7	1
11	6	1
12	1	1
13	3	1
14	2	1
15	4	0
16	5	0
17	0	0

VPN	PPN	Valid
28	b	1
29	8	0
2a	f	1
2b	d	0
2c	e	1
2d	a	0
2e	9	1
2f	c	0

Suppose the TLB contains the following data.

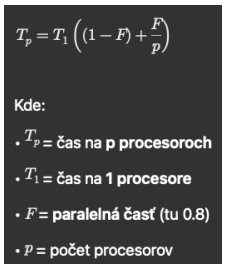
Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	4	7	1	5	2	1	a	b	1
1	4	6	1	5	4	0	a	8	0
2	4	1	1	5	5	0	a	f	1
3	4	3	1	5	0	0	a	d	0

Show how the virtual address `0x16fa` is translated to a physical address, by computing **VPN**, **TLB index**, **TLB tag**, **TLB hit**, **Page fault**, and **PPN**.

Provide values in hexademical notation (leaving out the leading `0x`), except where a true/false answer is requested (i.e. in the case of **TLB hit** and **Page fault**); provide **T**/**F** as an answer there instead. Write **NA** to indicate that there is no value to provide.

Question 33

Consider a program that takes 10 minutes to run on 4 processors, and which has an estimated parallel fraction F = 0.8.



A. How many minutes does this program take to run on 1 processor?

10 = t1((1-0.8)+(0.8/4)

10 = t1 (0.2+0.2)

10 = 0.4t1t

T1 = 10/0.4 = 100/4 = 25min trva run na 1 procesory.

Suppose the execution time of the non-parallelizable part of the workload can be reduced to 3 minutes.

B. How many minutes does such a version of the program take to run on 4 processors?

T1 bude iny, pretoze non-parallezable part is 3min pri 4 CPU.

1-F = 0.2. takze 0.2t1 = 3. Takze t1 = 15

T4 = (dopln vsetko do vzorca zas…. 15(0.2+0.8/4))

A mas 6

SECURITY LATTICE

Sposob ako urcit kam mozu tiect data. DATA MOZU TIECT V TABULKE IBA NAHOR – na rovnaku alebo vyssiu uroven bezpecnosti.

Pouziva sa v MPS – mandatory protection system (teda kde pravidla neurcuje pouzivatel ale su dane systemom)

A. *CONFIDENTIALITY – dovernost*

Chrani pred unikom informacii. Ma 2 urovne: secret a public.

B. *INTEGRITY – integrita – chrani pred poskodenim alebo podvrhnutim dat.*

Tiez ma 2 urovne: Truested and Untrusted.

Z toho sa robi product lattice

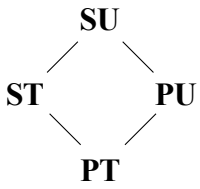
Znacka	*Vyznam*
SU	*Secret+Untrusted*
ST	*Secret+Trusted*
PU	*Public+Untrusted napr. Document ktory zdielame, ale nechceme aby ho poskodil nejaky virus.*
PT	*Public+Trusted*

Ak mas r-reading tak data tecu zo SUBORU do PROCESU

Ak mas writing – data tecu z procesu do subor.

Question 34

Consider the following three lattices of security levels.



product lattice

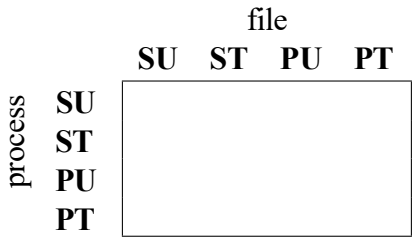
confidentiality lattice

integrity lattice

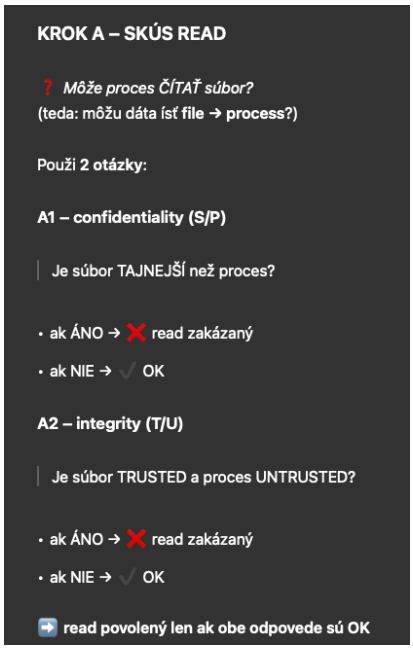
A security lattice states that data of a given security level can only flow to a storage which has a security level at or above the security level of the data. The confidentiality lattice, with its secret (**S**) and public (**P**) security levels, thus states that data must be protected from undesired disclosure. The integrity lattic

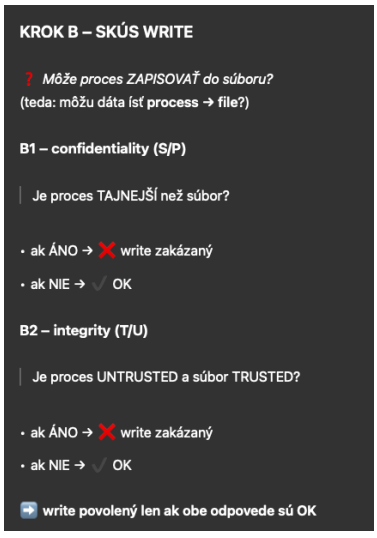
with its untrusted (**U**) and trusted (**T**) security levels, states that data must be protected from undesired destruction. The product lattice states both at the same time. For instance, a file labeled **PU** contains public data that is trusted (e.g. that we are sharing, but wish to protect from corruption by untrusted data).

Here we consider a Mandatory Protection System (MPS) based on labels drawn from this product lattice. Fill out (on the answer sheet) the first two rows of the access matrix for this MPS. Each field can either be blank (indicated by `-`), `r` , `w` , or `rw`.



End of hands-on questions.





Analytical Questions

Question 35

Why is it advised to strive for a stride-1 memory access pattern in your code?

Provide an answer on the answer sheet. We expect up to three sentences.

Question 36

How does “fetch-and-add” help us implement mutexes?

Provide an answer on the answer sheet. We expect up to three sentences.

End of analytical questions.

End of exam.